

Contents

1 Overview 2

1.1 Place in course 2

1.2 Learning Objectives 2

1.3 Equipment 2

2 Pre-Lab 2

3 Design Challenge 2

3.1 Multiple I2C devices 2

3.2 Tilt Detection 3

3.3 Orientation 4

3.4 Going Further (optional) 5

Deliverables

1 Device Driver (5 points) 2

2 Multiple I2C devices (5 points) 2

3 Computed Tilt (8 points) 4

4 Accuracy and Program Size (2 points) 4

5 Orientation (5 points) 4

1 Overview

1.1 Place in course

1.2 Learning Objectives

- Implement a device driver for an I2C device
- Manage two devices on the same I2C bus
- Use an accelerometer to measure tilt

1.3 Equipment

- Nucleo / breadboard setup
- Four digit seven-segment display
- Adafruit LSM303AGR accelerometer breakout
- Stemma-QT cable (4 wire)
- LEDs and 330 Ω resistors, as needed

2 Pre-Lab

1. Review the class materials about I2C/TWI, the accelerometer, and tilt sensing.
2. Complete the accelerometer device driver and acquire raw acceleration data from the sensor (Thursday's class).

Deliverable 1: Device Driver (*5 points*)

Write a clean, well orgnized device driver for the accelerometer (Thursday's class). Submit your `.c` and `.h` files.

3 Design Challenge

Wire the accelerometer breakout to your breadboard as shown in Figure 1¹.

The accelerometer and the seven-segment display share the I2C bus. The SDA terminals of the display and the breakout are connected, then wired to the Nucleo, and similarly for the SCL terminals. Leave the open space on the breadboard to add LEDs for part 2.

Connect the accelerometer using a “Stemma QT” cable (Figure 2). This connector is commonly used to connect I2C devices together.

3.1 Multiple I2C devices

This is the first time you have had a design which shares the I2C bus amongst multiple devices. To ensure that you are able to access both devices, write a program which reads the `WHO_AM_I_A` register from the accelerometer and writes the result to the 4 digit display. If your display does not show the correct value (where do you find the correct value?), you can connect your AD2 to the I2C signals and capture the traffic on the bus. As you saw in class, Waveforms has a built-in protocol decoder to help with this. This technique is known as bus sniffing and can be an extremely powerful debugging tool.

Deliverable 2: Multiple I2C devices (*5 points*)

¹Image modified from original at <https://www.adafruit.com/product/4209>

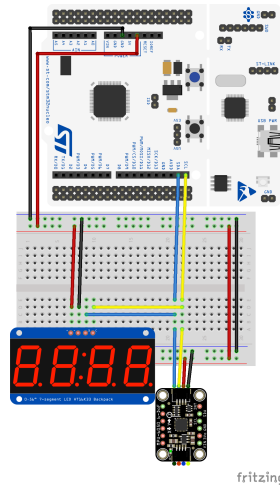


Figure 1: Sample Breadboard

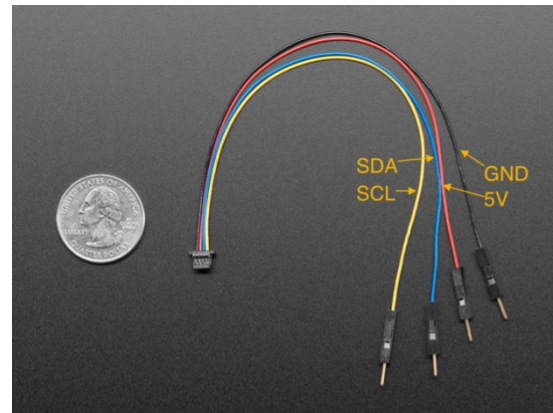


Figure 2: Stemma Connector

Submit your well commented code for this challenge. Obtain a signature confirming the value you read from the WHO_AM_I register.

Signature _____

3.2 Tilt Detection

From the application note (AN-1057), you know that tilting the sensor causes the gravity vector, $\mathbf{g}$, to be projected onto the x , y , and z axes attached to the sensor, and the tilt angle may be computed from the acceleration data. Tilt is the angle of inclination, Θ .

In Figure 3, the tilt angle is computed as $\Theta = \arctan(A_x/A_y)$, where A_x is the acceleration due to gravity along the X axis and A_y is the acceleration along the Y axis. The units of acceleration don't matter here because the ratio A_x/A_y is dimensionless.

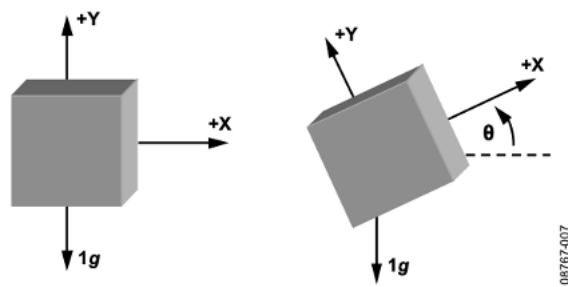


Figure 7. Two Axes Used for Tilt Sensing

Figure 3: Measuring Tilt

There are good polynomial approximations that may be employed if program memory is limited, but our programs have so far not taxed the memory of our system, so we'll use floating point arithmetic for this example.

There are two arctangent functions in C, `atan()` and `atan2()`. Use `atan2()`. It will locate the angle in the

proper quadrant and enable you to calculate tilt over the full 360° range. You can read about `atan2()` online². The result from `atan2()` is in radians. To convert radians to degrees, multiply by $180/\pi$. Add the line `#include <math.h>` in your source code to access the trig functions.

Write a program to compute and display the tilt of the sensor, from -180° to 180° . Be careful that your calculations take into account the orientation of the axes marked on the breakout board. Use the `round()` function to convert your computed tilt back to an integer for `printf()` and for the seven-segment display. Print the acceleration components and the tilt angle on your screen, and also display the tilt on the seven-segment display (with negative sign for negative angles).

Your phone also has a three-axis accelerometer, and there are many apps that can display tilt. Use one of these to check how well your tilt meter is working. There may be a 90° difference between what the phone reads and what your system gives, because the accelerometer in the phone may be rotated relative to the one on your breakout board. Hold the accelerometer against your phone and compare the readings.

Deliverable 3: Computed Tilt (8 points)

Submit your well commented code to Canvas. Obtain a signature confirming that your code behaves as expected.

Signature _____

Deliverable 4: Accuracy and Program Size (2 points)

How well does your computed tilt agree with your phone?

How large is the program that is loaded into the flash memory on your Nucleo?

Answer: _____

3.3 Orientation

Your phone is able to present the image on the screen “right side up” no matter how the phone is oriented. Suppose your sensor breakout board is a phone. Add four LEDs in a square array on the breadboard representing the two “portrait” orientations and the two “landscape” orientations. Use the acceleration / tilt data from the sensor to show, via the appropriate LED, which way it is oriented. Figure 4 shows the 4 orientations and the names Apple assigns to them for IOS (interesting historical note – Android considers the landscapes to be the opposite of those mentioned here. Everyone agrees about “Upside Down”). Your LEDs should track the top of the “phone”. Note: The sensor board is designed so that its X axis is along the longer dimension as shown in the figure. Keep this in mind when deciding how to interpret the acceleration data.

Deliverable 5: Orientation (5 points)

Submit your well commented code to Canvas. Also obtain a signature confirming that your code works as expected.

²https://www.tutorialspoint.com/c_standard_library/c_function_atan2.htm

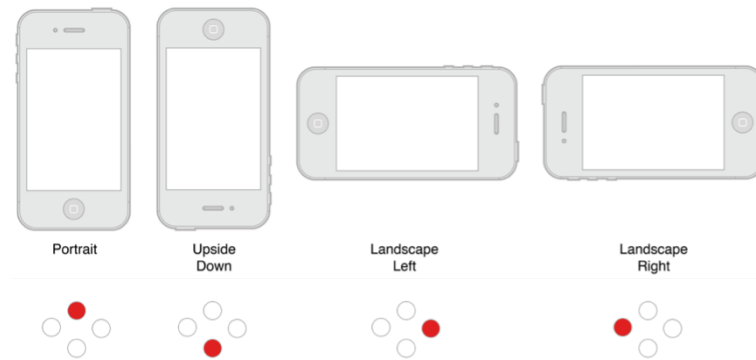


Figure 4: Phone Orientation

Signature _____

3.4 Going Further (optional)

Using all three axes, three angles may be computed that orient the accelerometer (hence, the breakout board) in space. The application note explains the geometry and mathematics of this calculation. Implement the calculations and display the angles (you'll probably want to use the serial port, but if you figure out a clever way to use the seven-segment display, do it!). Be careful that your results are correct, especially minus signs!